ALGORITHM

Comparison of Two General Trees

Algorithm: Construct and Compare Two Trees

1. Start.
2. Read the number of nodes for Tree 1 and create a node for each input value.
3. Read the child IDs for every node; treat node 1 as the root.
4. Validate that Tree 1 has one root, each other node has exactly one parent, all nodes are reachable, and there are no cycles.
5. Repeat Steps 2–4 to construct and validate Tree 2.
6. Call SameTree(root1, root2).
7. If both pointers are NULL, return true. If only one is NULL, return false.
8. Compare the values of the two current nodes. If different, return false.
9. Compare the number of children. If different, return false.
10. For each child position i, recursively compare child i of Tree 1 with child i of Tree 2. If any comparison is false, return false.
11. If all corresponding child comparisons succeed, return true.
12. Display the comparison result, release allocated memory, and end.

Pseudocode

```
SAME_TREE(A, B)
  IF A is NULL OR B is NULL
    RETURN (A and B are both NULL)
  IF A.value != B.value
    RETURN false
  IF number of children of A != number of children of B
    RETURN false
  FOR i = 0 TO childCount(A) - 1
    IF SAME_TREE(A.child[i], B.child[i]) is false
      RETURN false
  RETURN true
```

Important note

This algorithm compares child positions in order. If your assignment considers children unordered, an additional matching strategy is needed.